Comparing Function Approximation Methods and State Representations for Reinforcement Learning in Snake

Kiran Sairam Bethi Balagangadaran
Khoury College of Computer Sciences
Northeastern University
bethi.k@northeastern.edu

Abstract

We investigate how different state representations interact with function approximation methods in reinforcement learning, using the game Snake as a testbed. We compare three function approximation approaches — hand-crafted linear features, hash-based tile coding, and a Double DQN neural network with experience replay — across three state representations of increasing complexity (11, 109, and 126 dimensions), each trained for 20,000 episodes across 5 random seeds. Our experiments show that the neural network achieves the strongest and most consistent performance across all representations, reaching a mean score of 29.05 on extended features and exceeding the hand-coded greedy heuristic (22.89). Tile coding achieves 22.69 on compact features but collapses with dimensionality (3.21 on local, 1.32 on extended) due to hash collisions. Linear function approximation learns danger avoidance but not food-seeking, scoring below 4.0 across all representations. These results demonstrate that function approximation method and representation complexity interact strongly: tile coding is an efficient middle ground for well-designed low-dimensional features, while neural networks generalize robustly across representations when stabilized by Double DQN and experience replay.

Introduction

Reinforcement learning (RL) has achieved remarkable success in game-playing domains, from TD-Gammon's near-expert backgammon play (Tesauro 1995) to DQN's human-level Atari performance (Mnih et al. 2015). A recurring theme in this progression is the interaction between state representation and function approximation: the choice of how to represent the state, and how to approximate the value function over that representation, jointly determine whether an agent can learn effectively.

This interaction is well-studied at the extremes — tabular methods on small state spaces and deep neural networks on raw pixel inputs — but the middle ground is less explored. When is a simple linear model sufficient? When does automatic feature construction (tile coding) outperform hand-crafted features? At what point does nonlinear approximation become necessary? These questions have practical implications for domains where deep RL is overkill but tabular methods are insufficient.

We study these questions using the classic game Snake as a testbed. Snake is well-suited for this investigation because it offers a natural spectrum of state representations: from a minimal 11-dimensional binary feature vector (danger signals and food direction) to a rich 126-dimensional vector combining a local grid window with continuous distance features. The game's difficulty also increases within an episode as the snake grows, creating a non-stationary effective state space that challenges all methods.

Our contributions are as follows. First, we present a systematic comparison of three function approximation methods — hand-crafted linear features, hash-based tile coding, and a Double DQN neural network with experience replay — across three state representations of increasing complexity. Second, we demonstrate that tile coding dramatically outperforms hand-crafted linear features on the compact representation (22.69 vs. 0.80), highlighting the value of automatic feature construction. Third, we show that Double DQN with experience replay achieves the highest overall score (29.05 on extended features), exceeding the greedy heuristic by 27% and maintaining low variance across all three representations. Fourth, we provide evidence that the choice of function approximation method interacts strongly with representation complexity: tile coding degrades monotonically with dimensionality while neural networks do not.

Background

Temporal-Difference Learning

Temporal-difference (TD) learning (Sutton 1988) updates value estimates using the difference between successive predictions, without waiting for episode completion. For control, SARSA (Rummery and Niranjan 1994) updates the action-value function using the transition $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right] \quad (1)$$

where α is the learning rate and γ is the discount factor. SARSA is on-policy: it evaluates and improves the policy currently being followed, including its exploratory actions.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

Function Approximation

When the state space is too large for tabular methods, the value function must be approximated. In **linear function approximation**, the action-value function is:

$$\hat{q}(s, a; \mathbf{w}) = \mathbf{w}^\top \mathbf{x}(s, a) \tag{2}$$

where $\mathbf{x}(s, a) \in \mathbb{R}^d$ is a feature vector and $\mathbf{w} \in \mathbb{R}^d$ is a weight vector. The semi-gradient SARSA update becomes:

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta_t \, \mathbf{x}(S_t, A_t) \tag{3}$$

where $\delta_t = R_{t+1} + \gamma \hat{q}(S_{t+1}, A_{t+1}; \mathbf{w}) - \hat{q}(S_t, A_t; \mathbf{w})$.

Tile coding (Sutton and Barto 2018; Sherstov and Stone 2005) automatically constructs binary feature vectors. Multiple overlapping tilings partition the normalized state space, and the feature vector has one active bit per tiling. The value function is the sum of weights at active tile indices. Nearby states share most active tiles, providing smooth generalization.

Neural network approximation replaces the linear model with a nonlinear function parameterized by $\boldsymbol{\theta}$, updated via backpropagation. **Double DQN** (van Hasselt, Guez, and Silver 2016) addresses the overestimation bias of vanilla DQN by decoupling action selection from evaluation: the online network selects the greedy next action $\hat{a} = \arg\max_a Q(S_{t+1}, a; \boldsymbol{\theta})$, while a periodically-synchronized target network scores it:

$$Y_t = R_{t+1} + \gamma Q(S_{t+1}, \hat{a}; \boldsymbol{\theta}^-) \tag{4}$$

Combined with a replay buffer that breaks temporal correlations in the training data, this approach stabilizes learning with nonlinear function approximation.

The Deadly Triad

Sutton and Barto (2018) identify the *deadly triad*: the combination of function approximation, bootstrapping, and off-policy learning can cause divergence. Tsitsiklis and Van Roy (1997) proved that on-policy TD with linear function approximation converges with probability 1, while Baird (1995) showed that off-policy TD with linear FA can diverge even in simple cases. Van Hasselt et al. (2018) confirmed these risks empirically in deep RL settings.

Our linear and tile coding agents use on-policy semi-gradient SARSA, maintaining the full convergence guarantee. Our neural network agent departs from on-policy sampling via experience replay, entering the off-policy leg of the deadly triad; we mitigate the resulting instability with Double DQN (reduces overestimation bias) and a hard-synchronized target network (stabilizes regression targets), following established practice in deep RL (Mnih et al. 2015; van Hasselt, Guez, and Silver 2016).

Related Work

Snake as an RL Domain

Snake has been studied with a range of RL methods. Wei et al. (2018) applied DQN with CNN processing of raw pixel frames, introducing dual experience replay and a training gap strategy. Bonnici et al. (2022) compared Q-learning, SARSA, PPO, and A* across vector, CNN, and raycasting representations, finding PPO with raycasting performed best. Sebastianelli et al. (2021) showed that a sensor-based 11-value state representation trains far more easily than CNN approaches. A Stanford CS230 project by Jiang and Mai compared an 11-boolean feature vector against grayscale screenshots, finding the compact features achieved sevenfold higher scores (Jiang and Mai 2021). Ma et al. (2016) found that SARSA outperformed Q-learning on Snake with 128 discrete states and proved that optimal Snake play involves NP-hard subproblems.

Linear FA vs. Neural Networks

Liang et al. (2016) demonstrated that hand-crafted features with linear function approximation can match DQN performance on Atari games. Elfwing et al. (2018) compared TD(λ) with neural networks versus linear FA in Tetris, finding that neural networks significantly outperformed linear methods when the value function was inherently nonlinear. Tesauro (1995) showed that adding hand-crafted expert features to a neural network dramatically improved TD-Gammon's performance.

Positioning

No prior work has systematically compared function approximation methods — specifically hand-crafted linear features, tile coding, and a neural network — across multiple state representations for Snake using a common experimental framework. Most existing studies use off-policy DQN and focus on the neural network side. We fill this gap by holding the training framework fixed and varying both the approximation method and the representation.

Methods

Environment

We implement a custom Snake environment on a 20×20 grid following the Gymnasium interface (Farama Foundation 2024). The snake starts at the center with length 3, facing right. The action space is $\mathcal{A} = \{\text{STRAIGHT}, \text{LEFT}, \text{RIGHT}\}$ relative to the current heading, eliminating the possibility of reversing into the snake's body. Rewards are $+10$ for eating food, -10 for dying (wall or self-collision), and -0.1 per step to encourage efficiency. The discount factor is $\gamma = 0.95$. Episodes terminate on death or after $3 \times \text{grid_size}^2 = 1{,}200$ steps (truncation).

For the neural network agent, we additionally apply potential-based reward shaping (Ng, Harada, and Russell 1999): a bonus of $F(s, s') = 0.5 \times (d(s) - d(s'))$ is added each step, where d is the Manhattan distance from the snake's head to the food. This bonus is omitted when food is eaten, as the food position resets. Potential-based shaping does not alter the optimal policy (Ng, Harada, and Russell 1999) but provides a denser reward signal that speeds convergence.

State Representations

We compare three representations of increasing complexity.

Compact (11 dimensions): Three binary danger signals (wall or body one step ahead in each relative direction), four one-hot direction bits, and four binary food-direction indicators. This matches the standard design used in most Snake RL implementations (Loeber 2020; Zhou 2023).

Local Neighborhood (109 dimensions): A 5×5 grid window centered on the snake's head, with each cell one-hot encoded as [empty, food, wall, body] (100 features), plus food direction (4) and snake length bucket (5).

Extended (126 dimensions): The local neighborhood features augmented with normalized Manhattan distance to food (1), distance to the nearest body segment in each cardinal direction (4), distance to walls (4), normalized snake length (1), current direction one-hot (4), and explicit danger signals (3).

For the linear and tile coding models, state-action features are constructed by one-hot action stacking: the state vector is placed in the block corresponding to the action, with zeros elsewhere, yielding feature dimensions of 33, 327, and 378 respectively.

Algorithms

All three algorithms use ε-greedy exploration with ε decaying linearly from 1.0 to 0.01 over 80% of the 20,000 training episodes.

Linear FA SARSA. The update follows Equation 3 with learning rate $\alpha = 0.01$.

Tile Coding SARSA. We use overlapping tilings with 4 tiles per dimension and a hash table of 262,144 entries. We use 8 tilings for the compact representation and 4 tilings for local and extended representations (where additional tilings provide diminishing returns given the fixed hash table size). The effective learning rate is $\alpha = 0.05/n_{\text{tilings}}$ per the standard convention (Sutton and Barto 2018). Feature normalization ranges are estimated by sampling 1,000 random episodes before training.

Double DQN. A two-layer network with hidden dimensions (256, 128) and ReLU activations outputs Q-values for all three actions simultaneously. We use a replay buffer of capacity 50,000 transitions, sampling mini-batches of 64 transitions per gradient step. A target network is hard-copied from the online network every 100 episodes. The training target uses Double DQN as described in the Background section: the online network selects the greedy next action and the target network evaluates it, reducing overestimation bias. The loss is Huber (smooth ℓ_1) for robustness to large TD errors. We use the Adam optimizer (Kingma and Ba 2015) with learning rate 10^{-3} and gradient clipping at max norm 5.0. Distance-based reward shaping is applied during training.

Baselines

We compare against two baselines: a **random policy** (uniform random actions) and a **greedy heuristic** that avoids immediate danger and moves toward food by Manhattan distance.

Table 1: Tabular Q-learning on a 10×10 grid (3,000 episodes). Richer representations cause state space explosion.

Representation	Dims	States	Q-entries	Score
Compact	11	253	694	9.00
Local	109	7,443	13,649	1.93
Extended	126	28,160	41,909	0.40

Table 2: Final performance (mean ± std over the last 1,000 episodes, 5 seeds) on a 20×20 grid after 20,000 training episodes. Bold marks the best score per column.

	Compact	**Local**	**Extended**
Linear FA	0.80 ± 0.07	0.67 ± 0.03	3.90 ± 0.98
Tile Coding	22.69 ± 0.54	3.21 ± 0.17	1.32 ± 0.03
Double DQN	$\mathbf{24.26 \pm 0.39}$	$\mathbf{22.51 \pm 0.38}$	$\mathbf{29.05 \pm 0.32}$

Experiments

Experimental Setup

We run 9 configurations (3 algorithms × 3 representations), each for 20,000 episodes across 5 random seeds. Performance is measured by the average score (food eaten per episode) over the final 1,000 episodes. We report mean and standard deviation across seeds. Learning curves are smoothed with a 100-episode moving average.

Tabular Methods Break Down

We first demonstrate the motivation for function approximation by running tabular Q-learning on all three representations using a 10×10 grid (3,000 episodes). Table 1 shows the results.

The compact representation has only 253 unique states and achieves strong performance (9.00). The local neighborhood encounters 7,443 unique states, most visited only once, and performance drops to 1.93. The extended representation explodes to 28,160 unique states with near-random performance (0.40 vs. 0.15 for a random policy). This confirms that function approximation is essential for richer representations.

Baseline Performance

On the 20×20 grid (200 evaluation episodes), the random policy scores 0.15 ± 0.38 (max 2), dying primarily from wall collisions. The greedy heuristic scores 22.89 ± 6.00 (max 33), dying primarily from self-collision where it chases food greedily without planning around its own body. These establish lower and upper reference points.

Main Results

Table 2 presents the final performance across all 9 configurations after 20,000 training episodes.

Analysis

Representation and Approximation Interaction. The results reveal a strong interaction between representation

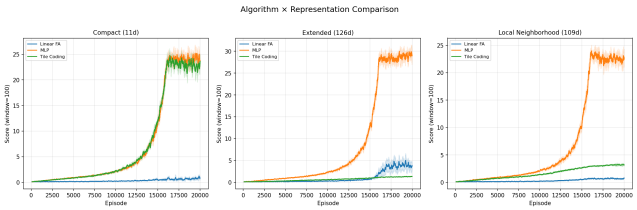

Figure 1: Learning curves across all 9 configurations grouped by representation. Shaded regions show ± 1 std across 5 seeds.

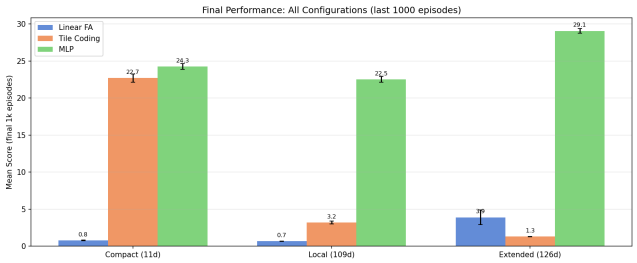

Figure 2: Final performance bar chart across all configurations, grouped by representation.

complexity and approximation method, summarized in Figure 3. Tile coding achieves its best score on compact features (22.69), competitive with the greedy heuristic (22.89), but collapses to 3.21 on local and 1.32 on extended. Double DQN shows the opposite pattern: it achieves its highest score on extended features (29.05), exceeding the greedy heuristic by 27%, and wins on every representation (24.26 compact, 22.51 local), including outperforming tile coding on compact (22.69) where tile coding is strongest. Linear FA is weak everywhere, peaking at 3.90 on extended.

This interaction aligns with theoretical expectations and is visible directly in the learning curves of Figure 1 and the bar chart of Figure 2. Tile coding builds generalization through spatial overlap in the tiled state space; this structure is well-defined in low dimensions but breaks down under hash collisions at high dimensionality. Neural networks learn generalization directly from data via gradient descent, making them more robust to representation choice when given sufficient training and a stable learning algorithm.

Tile Coding vs. Hand-Crafted Features. Tile coding outperforms hand-crafted linear features by a factor of $28\times$ on the compact representation (22.69 vs. 0.80). Tile coding's overlapping tilings create a much richer feature space than the 33-weight linear model: nearby states share most active tiles, enabling smooth value function generalization that binary features cannot provide.

However, tile coding's advantage disappears with higher-dimensional representations. With 109 or 126 input dimensions, the hash-based coder maps an astronomically large index space into 262,144 buckets, causing frequent collisions. Genuinely different states activate the same tile indices, corrupting the value function. Tile coding's scores drop monotonically with dimensionality: $22.69 \rightarrow 3.21 \rightarrow 1.32$ — a $17\times$ reduction from compact to extended. This is a known limitation of hash-based tile coding (Sherstov and Stone 2005) and highlights the necessity of dimensionality reduction or feature embedding before applying hash-based linear approximation in high-dimensional grid-world environments.

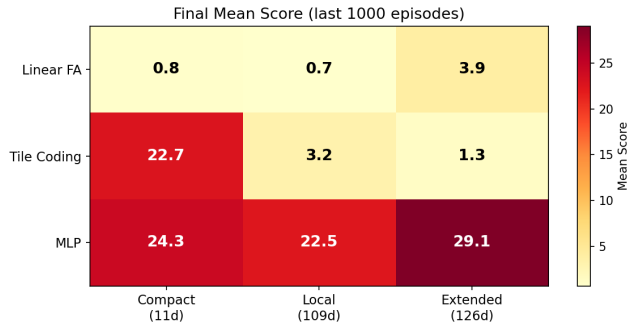

Figure 3: 3×3 heatmap of mean scores over the final 1,000 episodes (algorithms $\times$ representations). Double DQN dominates all cells; tile coding is competitive only on compact.

Double DQN Stability Across Representations. One of the most striking results is Double DQN's uniformly low variance: ±0.39, ±0.38, and ±0.32 on compact, local, and extended representations respectively. This stability is not accidental. Experience replay breaks the temporal correlations that make sequential on-policy updates noisy, providing a more stationary regression problem. Double DQN's decoupled target reduces the overestimation feedback loop that destabilizes vanilla DQN. The hard-synchronized target network, copied every 100 episodes, prevents runaway value estimates by holding the regression target fixed within each update window.

The result is that Double DQN generalizes reliably even on the high-dimensional sparse local representation (22.51), where pure on-policy nonlinear function approximation typically struggles (van Hasselt et al. 2018). This supports the view that for neural network function approximation in game environments, off-policy training infrastructure (replay + target network) is essential for consistent learning.

Weight Analysis for Linear FA. The linear model's learned weights on the compact representation reveal interpretable structure. The dominant weights are the danger signals: the weight for "danger straight" in the STRAIGHT action block reaches approximately -8, and the analogous danger weights in the LEFT and RIGHT blocks follow the same pattern. The agent has learned a strong collision-avoidance policy.

However, food-direction weights are comparatively small and noisy (< 2 in magnitude). The linear model over 11 binary features can express avoidance but lacks expressiveness for nuanced food-seeking. Under a near-greedy policy ($\varepsilon \approx 0.01$), the agent enters deterministic loops because the compact representation maps many distinct game states to identical feature vectors, creating action cycles.

Qualitative Behavior. The most striking contrast is between tile coding and linear FA on the compact representation. The tile coding agent navigates efficiently toward food, makes deliberate turns to avoid its own body, and consistently reaches scores above 20 (individual seeds reach a maximum of 63). The linear FA agent learns to avoid immediate collisions but exhibits repetitive looping, oscillating between two directions because the 11 binary features collapse distinct game states to the same vector.

The Double DQN agent on extended features displays the most sophisticated behavior: navigating toward food while maintaining spatial awareness of its body, occasionally taking longer detours to avoid self-trapping. Individual seeds reach maximum scores of 66, representing near-expert-level play. On the local representation, behavior is consistent across seeds (± 0.38), further confirming the stabilizing effect of experience replay.

Code and Supplementary Material. The complete source code for the custom environment, training algorithms, and evaluation scripts, alongside gameplay recordings for all configurations, are publicly available at:

`https://github.com/Kiran1510/snake-rl`.

Conclusion

We presented a systematic comparison of three function approximation methods across three state representations for learning to play Snake, using 20,000 training episodes and 5 random seeds. The key findings are: (1) Double DQN with experience replay achieves the highest and most consistent performance, reaching 29.05 on extended features and exceeding the greedy heuristic (22.89) by 27%; (2) tile coding is an effective middle ground for compact, well-designed features (22.69) but collapses monotonically with dimensionality due to hash collisions; (3) linear FA learns danger avoidance but not food-seeking, remaining below 4.0 across all representations.

These results yield three practical insights. First, tile coding is a powerful and underappreciated middle ground: when features are low-dimensional and well-designed, it nearly matches neural network performance at a fraction of the computational cost. Second, representation design matters as much as algorithm selection: Double DQN scores 29.05 on extended but 22.51 on local — a 23% variation driven purely by representation, not algorithm. Third, the stabilizing infrastructure of Double DQN and experience replay is critical for consistent neural network performance across diverse representations.

Limitations include the focus on a single domain (Snake), moderate hyperparameter tuning, and the absence of prioritized experience replay (Schaul et al. 2016), which may further improve sample efficiency. Future work should investigate adaptive tile coding (Sherstov and Stone 2005) for high-dimensional inputs and continuous ray-casting representations to provide smoother gradients for function approximators.

References

Baird, L. 1995. Residual algorithms: Reinforcement learning with function approximation. In *Proceedings of the 12th International Conference on Machine Learning*, 30–37.

Bonnici, N.; Vassallo, C.; and Valentino, G. 2022. Exploring reinforcement learning: A case study applied to the popular snake game. In *Lecture Notes in Networks and Systems*, volume 382, 169–192. Springer.

Elfwing, S.; Uchibe, E.; and Doya, K. 2018. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks* 107:3–11.

Farama Foundation. 2024. Gymnasium documentation. `https://gymnasium.farama.org`.

Jiang, J., and Mai, M. 2021. Snakeai. Stanford CS230 Project Report.

Kingma, D. P., and Ba, J. 2015. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.

Liang, Y.; Machado, M. C.; Talvitie, E.; and Bowling, M. 2016. State of the art control of atari games using shallow reinforcement learning. In *Proceedings of the 15th International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, 485–493.

Loeber, P. 2020. snake-ai-pytorch. GitHub repository.

Ma, B.; Tang, M.; and Zhang, J. 2016. Exploration of reinforcement learning to snake. Stanford CS229 Project Report.

Mnih, V.; Kavukcuoglu, K.; Silver, D.; Rusu, A. A.; Veness, J.; Bellemare, M. G.; Graves, A.; Riedmiller, M.; Fidjeland, A. K.; Ostrovski, G.; et al. 2015. Human-level control through deep reinforcement learning. *Nature* 518(7540):529–533.

Ng, A. Y.; Harada, D.; and Russell, S. 1999. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 278–287.

Rummery, G. A., and Niranjan, M. 1994. On-line q-learning using connectionist systems. Technical Report CUED/F-INFENG/TR 166, Cambridge University Engineering Department.

Schaul, T.; Quan, J.; Antonoglou, I.; and Silver, D. 2016. Prioritized experience replay. In *Proceedings of the 4th International Conference on Learning Representations (ICLR)*.

Sebastianelli, A.; Tipaldi, M.; Ullo, S. L.; and Ullo, A. 2021. A deep q-learning based approach applied to the snake game. In *Proceedings of the IEEE International Conference on Signal Processing*.

Sherstov, A. A., and Stone, P. 2005. Function approximation via tile coding: Automating parameter choice. In *Proceedings of the 6th International Symposium on Abstraction, Reformulation and Approximation (SARA)*, 194–205. Springer.

Sutton, R. S., and Barto, A. G. 2018. *Reinforcement Learning: An Introduction*. MIT Press, 2nd edition.

Sutton, R. S. 1988. Learning to predict by the methods of temporal differences. *Machine Learning* 3(1):9–44.

Tesauro, G. 1995. Temporal difference learning and td-gammon. *Communications of the ACM* 38(3):58–68.

Tsitsiklis, J. N., and Van Roy, B. 1997. An analysis of temporal-difference learning with function approximation. *IEEE Transactions on Automatic Control* 42(5):674–690.

van Hasselt, H.; Doron, Y.; Strub, F.; Hessel, M.; Sonnerat, N.; and Modayil, J. 2018. Deep reinforcement learning and the deadly triad. *arXiv preprint arXiv:1812.02648*.

van Hasselt, H.; Guez, A.; and Silver, D. 2016. Deep reinforcement learning with double q-learning. In *Proceedings of the 30th AAAI Conference on Artificial Intelligence*, 2094–2100.

Wei, L.; Wang, Z.; Yang, B.; and Liu, S. 2018. Autonomous agents in snake game via deep reinforcement learning. In *Proceedings of the IEEE International Conference on Agents (ICA)*, 79–82.

Zhou, N. 2023. Teaching an ai to play the snake game using reinforcement learning. Medium article.